Decision-Making Decisions: Artificial Intelligence In Computer Games

George Britton

June 2023

Abstract

Artificial intelligence (AI) is widely used in various applications, including computer games, to create believable and immersive experiences. This paper examines three types of AI used in computer games: Goal-Oriented Action Planning, Finite-State Machines, and Behaviour Trees. The workings, strengths, and weaknesses of each type are explained, and a comparison is made by analysing their implementation in three different games: Hitman, Skyrim, and DEFCON.

The findings reveal that Goal-Oriented Action Planning is a cost-effective approach for creating believable AI in Hitman, while Finite-State Machines are suitable for the open-world setting of Skyrim, and Behaviour Trees aid in developing diverse behaviours in DEFCON.

Electronic copy available at: https://ssrn.com/abstract=4513757

Contents

1 Introduction **3**
- 1.1 Key Issues . . . 3
- 1.2 Aim . . . 3

2 Literature Review **3**

3 Methods **5**
- 3.1 Goal-Oriented Action Planning . . . 5
  - 3.1.1 Goal Selection . . . 5
  - 3.1.2 Formulation . . . 5
  - 3.1.3 Discrepancy Detection . . . 6
  - 3.1.4 Action Execution . . . 6
  - 3.1.5 Payoff . . . 6
  - 3.1.6 Repeat . . . 6
- 3.2 Finite-State Machines . . . 7
  - 3.2.1 States . . . 7
  - 3.2.2 Events . . . 7
  - 3.2.3 Transitions . . . 8
- 3.3 Behaviour Trees . . . 8
  - 3.3.1 Composite Nodes . . . 8
  - 3.3.2 Decorator Nodes . . . 8
  - 3.3.3 Leaf Nodes . . . 9

4 Analysis **9**
- 4.1 Comparison . . . 10
- 4.2 Evaluation . . . 10

5 Conclusion **11**

Electronic copy available at: https://ssrn.com/abstract=4513757

1 Introduction

Artificial intelligence encompasses various approaches that handle the transition between different behaviours. It is applied in different domains such as computer games, robotics, and market analysis. This report focuses on three approaches to AI in computer games: Goal-Oriented Action Planning, Finite-State Machines, and Behaviour Trees. The aim is to explore which AI approach is most suitable for specific computer games, namely Hitman, Skyrim, and DEFCON.

1.1 Key Issues

This report addresses the question of which AI approach—Goal-Oriented Action Planning (GOAP), Finite-State Machines, or Behaviour Trees—is most suitable for different computer games. The suitability will be assessed based on the stealth-strategy game *Hitman* (Vorsholt 2016), the open-world role-playing game *Skyrim* (Howard 2011), and the tactical simulation game *DEFCON* (Delay 2006).

1.2 Aim

The aim of this report is to provide an understanding of how different AI applications can be used in computer game development. Relevant literature on the three types of AI will be reviewed, followed by an analytical comparison and confident determination of the most applicable AI implementations for the three mentioned games. The report will also identify the aspects of the games that can be optimised using these AI approaches.

2 Literature Review

There is a wealth of research currently available for review regarding the different approaches to artificial intelligence in computer games. One key paper is *Applying Goal-Oriented Action Planning to Games* (Orkin 2003), which discusses the use of GOAP in games and explains the mechanics of how it works. Orkin (2003) discusses the link between preconditions and effects resulting from an AI formulating a plan, and how it can create longer plans in order to achieve more complex goals. This article is a prevalent piece of literature in the field of computer game AI, as it explains in simple terms the techniques used in making GOAP function.

Lim et al. (2010) explains the use of behaviour trees and discusses the advantages over finite-state machines, such as the scalability it has in contrast to finite-state machines' exponentially growing complexity. It also talks about the construction of behaviour trees as actions and conditions, going further into details explaining the two main composite constructs: sequences and selectors, and the role of decorators and leaf nodes.

Electronic copy available at: https://ssrn.com/abstract=4513757

An article in the February 2004 edition of *ACMQueue* called *AI in Computer Games* (Nareyek 2004) explains how finite-state machines work. Figure 1 shows a basic diagram of how a finite-state machine might work in the instance of an enemy attempting to locate and attack the player. This figure, however, lacks the portrayal of the decisive logic between the events and transitions, and could better represent a finite-state machine by showing the decision logic that drives one state to *one of a choice of others*.

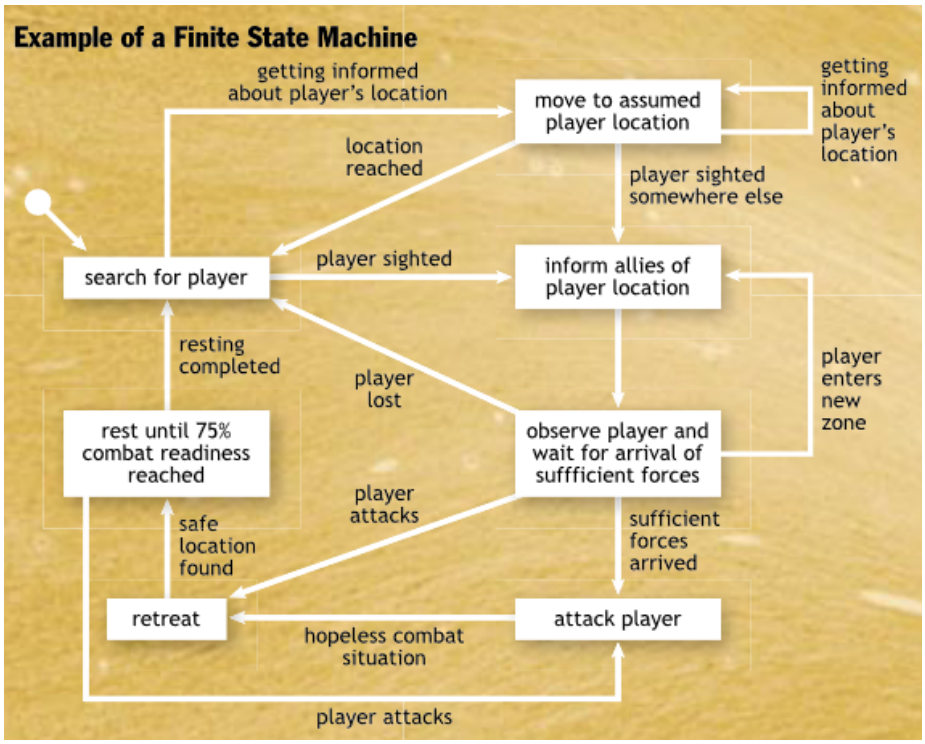


Figure 1: Depiction of Finite-State Machines in Nareyek (2004)

Lim et al. (2010) also mis-explains similarly, in that "finite-state machines in games tend to have states within states, multiple state variables, randomness in code executing every tick within a state." This does not need to be the case. By using the game object's inherent variables, such as the enemy AI's health or the shopkeeper's wealth, a finite-state machine could compare variables at the point of transition to make logical decisions regarding which state to transition to. An enemy AI with low health may behave differently upon seeing the player than an enemy AI with full health would. Lim et al. (2010) describes these as *Hierarchical* Finite-State Machines, as they group finite-state machines into smaller clusters and connect them using a larger finite-state machine that bridges the gap with events and variable conditions.

Electronic copy available at: https://ssrn.com/abstract=4513757

3 Methods

This section provides a detailed explanation of each AI approach and its practical uses. Each approach is examined individually, including its functionality, technical limitations, and its application in computer games.

3.1 Goal-Oriented Action Planning

GOAP is a "decision-based architecture" (Orkin 2003) in which an AI is provided with a list of different **actions** (defined as "a single atomic step within a plan" (Orkin 2003)) it can take, each with their own **preconditions** (conditions that must be true before the action can be executed) and **effects** (changes to the world resulting from the completion of the action). What sets GOAP apart from other approaches is its ability to formulate a **plan** by identifying a desired goal state and working towards it through a sequence of actions.

The decision-making process of a GOAP AI involves five primary steps:

3.1.1 Goal Selection

During this stage, the AI considers the possible conditions of the world and selects a condition it aims to achieve. For instance, an enemy AI might choose the goal of eliminating the player and set it as the target state.

3.1.2 Formulation

The next step is to examine the list of available actions and work backward from the goal to the initial state. The AI evaluates the current state of the goal and matches it against the effects of all possible actions. As an example, an appropriate action for the enemy AI could be "AttackPlayer," which would result in the player's death. The preconditions for the action, such as "CanSeePlayer" and "HasWeapon," are checked against the effects of other actions to determine which actions fulfil the most preconditions. This process continues until the preconditions for the initial action in the plan (represented as a queue of actions) are already satisfied, allowing the execution of the action queue.

The search is performed from goal to start because a start-to-finish search could lead to paths that reach dead ends before being able to backtrack and find the correct action. This search process is analogous to a depth-first search (Weik 2001). The backward search through actions can be accomplished using an A* search algorithm (Hart et al. 1968), which explores the state-space based on a heuristic (e.g., the number of unfilled preconditions plus the current length of the plan) to find the shortest path to the goal. This guarantees a valid path, provided one exists, as "the preconditions on the actions prevent the planner from formulating an invalid plan" (Orkin 2003).

Electronic copy available at: https://ssrn.com/abstract=4513757

3.1.3 Discrepancy Detection

During this stage, the AI examines the plan to ensure that the laid-out steps are still valid in sequence. If any inconsistencies are detected, the process restarts from the *Goal Selection* phase to create a new plan. If the plan remains valid, the AI proceeds to the next step. The *Discrepancy Detection* stage is only utilised after the first action is completed since the AI initially formulates a valid plan and can immediately start executing it without the need for validation.

3.1.4 Action Execution

In this stage, the AI executes the next action in the plan, removing it from the queue. The AI may iterate over multiple time steps, resembling a simplified state machine. For example, the enemy AI might have the action *GoToPlayer*, which concludes when the player enters the enemy's line of sight, prompting the AI to proceed to the next step.

3.1.5 Payoff

The *Payoff* stage is the final step of the action loop and occurs when an action is completed. In this step, the AI resolves the effects of the action, modifying the world state to reflect the action taken. In the case of our enemy example, when the player comes into the enemy's line of sight, the enemy completes the *GoToPlayer* action, resulting in the *CanSeePlayer* variable being set to true.

3.1.6 Repeat

The AI then repeats steps 3 (*Discrepancy Detection*), 4 (*Action Execution*), and 5 (*Payoff*) until completion, invalidation, or until a new goal becomes more relevant (Orkin 2003). This process can be terminated prematurely if the AI is deleted, such as when the enemy is killed or the player leaves the level.

Figure 2 illustrates the depiction of a final plan created using GOAP according to Orkin (2003). It showcases how the goal state (in this case, *PlayerDead* being true) and the initial state are connected through a queue of actions derived from a somewhat disorganised action base. Each action in the queue is expected to satisfy the preconditions of the subsequent action logically. For instance, the *Attack* action cannot be executed if the enemy has no ammunition in their weapon, and the *ReloadWeapon* action cannot be performed if the enemy does not have a weapon drawn.

Electronic copy available at: https://ssrn.com/abstract=4513757

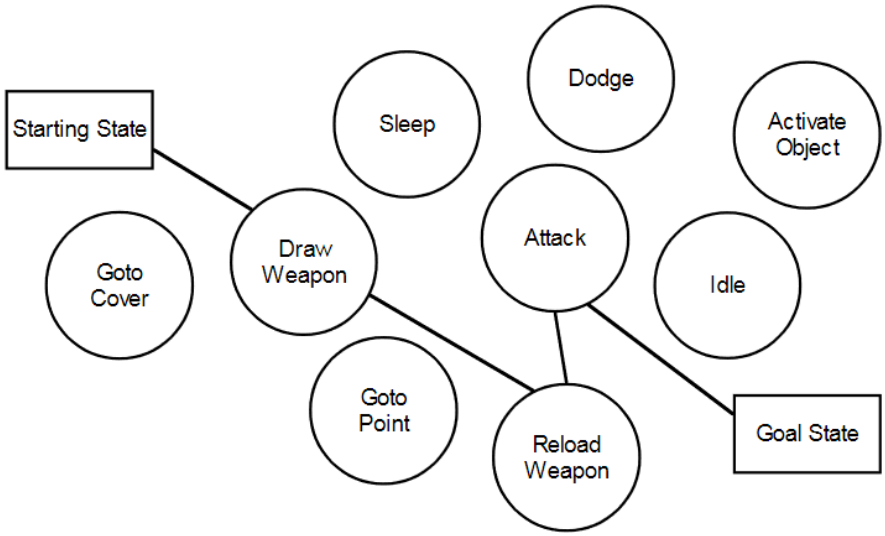


Figure 2: Action base and goal selection diagram from Orkin (2003)

3.2 Finite-State Machines

Finite-State Machines (FSMs) are a "reactive technique" (Weber et al. 2010) used in AI that consist of states, events, and transitions. They are widely recognised as the "most popular form of AI in modern computer games" (Sweetser & Wiles 2002) due to their ability to provide specific behaviours. At its core, an FSM divides an object's behaviour into logical states, where each state represents a distinct behaviour that the object can exhibit (Sweetser & Wiles 2002). For example, a state in an AI system could be the state of searching for the player, while an event might occur when the player is located, triggering a transition from the search state to the attack state. These states and transitions form a cyclic tree structure, where each state is a node and each transition is an edge.

3.2.1 States

A state represents a specific condition or behaviour in which the AI can be. For instance, a guard AI may be in a *Patrol* state, where it continuously walks along a predetermined circular path. This behaviour can either continue indefinitely or terminate, leading the AI to enter another state. The completion of the circular path, in this case, serves as an event that triggers a state transition.

3.2.2 Events

An event signifies a change in circumstances that influences the AI's behaviour. In the example of the guard AI, if it is in the *Patrol* state following a circular

Electronic copy available at: https://ssrn.com/abstract=4513757

path, it would continue indefinitely if there are no interruptions. However, if the player comes into view, the event of spotting the player would initiate a transition to a different behavioural state.

3.2.3 Transitions

A transition represents a shift from one behavioural state to another. The logic of the AI resides primarily in the transitions, as they determine which state to transition into based on specific events. For example, when a guard AI in the *Patrol* state spots the player, it may transition to the *Attack* state, *Flee* state, or even the *Investigate* state if it was only a brief glimpse. The conditions for these transitions are set by the developer, allowing the same guard AI to transition to the *Attack* state if it has a weapon and high health, but transition to the *Flee* state otherwise.

3.3 Behaviour Trees

Behaviour Trees (BTs) are a hierarchical approach to organising behaviours in a descending order of complexity (Perez et al. 2011). They are an AI method that constructs sequential plans of actions, which in turn affect the game world, and includes conditional queries about the game world itself. The resulting structure is a Directed Acyclic Graph (DAG) that can be dynamically modified to reflect the current state of the game. This approach allows developers to define a high-level goal and subsequently break it down into smaller sub-tasks that form the basis of the required behaviour trees (Lim et al. 2010). Behaviour trees consist of three primary node types: composite nodes, decorator nodes, and leaf nodes.

3.3.1 Composite Nodes

Composite nodes control the execution flow of the behaviour tree by selecting and executing child nodes. There are two main types of composite nodes: sequence nodes and selector nodes. Sequence nodes execute their children nodes in order until one fails, acting as a logical AND operation. On the other hand, selector nodes execute their children nodes in order until one succeeds, functioning as a logical OR operation (Perez et al. 2011).

3.3.2 Decorator Nodes

Decorator nodes are similar to composite nodes but differ in that they can have only one child. Their purpose is to modify or adapt their child's output to fit both the current world state and the goal of the tree. For example, a decorator node may alter the traversal of the graph by looping through nodes or negating a node's output. Alternatively, a decorator node may invert the output of its child, acting as a logical NOT operation (Perez et al. 2011).

Electronic copy available at: https://ssrn.com/abstract=4513757

3.3.3 Leaf Nodes

Leaf nodes are the terminal nodes of the tree branches. They represent either a condition or an action. A condition node queries the world state and returns a result to its parent. An action node makes decisions and carries out specific tasks. For example, a condition node might check if the player is visible (*CanSeePlayer*), while an action node might perform the action of moving towards the player (*GoToPlayer*) if the condition *CanSeePlayer* is false.

Figure 3 illustrates a basic behaviour tree presented in Lim et al. (2010). The tree demonstrates how a high-level goal (*Place Silos*) can be decomposed into sub-tasks. In this example, a condition node checks if the game state is at DEFCON 5, followed by an action node that calculates a random location and another action node that places a structure (a silo) at that location. This depiction showcases the breakdown of a composite goal node (*PlaceSilo*) into modular, atomic leaf nodes to achieve the desired behaviour.

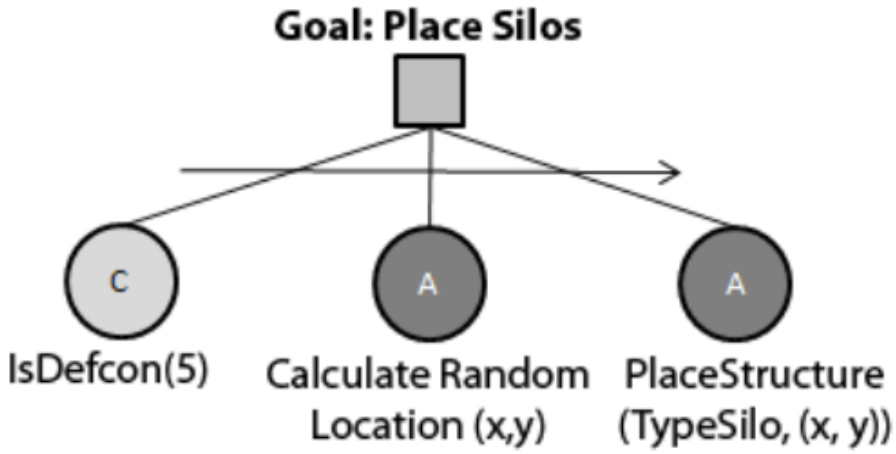


Figure 3: Basic behaviour tree depicted in Lim et al. (2010)

4 Analysis

In this section, I will examine the strengths and weaknesses of the AI approaches described earlier and compare them to the respective computer games. The objective is to determine the most suitable AI approach for each game and identify the specific areas within each game where these approaches would be most effective.

Electronic copy available at: https://ssrn.com/abstract=4513757

4.1 Comparison

GOAP offers several strengths. It allows AI to handle contingencies and adapt to changes without requiring direct human input. This flexibility is achieved by decoupling goal selection from goal execution logic, making the code easier to read and maintain (Weber et al. 2010). Additionally, GOAP can function effectively with standard search algorithms like A*, which helps minimise computational resource usage. However, it's important to note that GOAP does not eliminate the need for FSMs; rather, it simplifies the necessary FSM implementation (Orkin 2003).

FSMs, while not mutually exclusive with other AI implementations such as GOAP or neural networks, offer their own set of benefits. They are relatively simple to program, understand, and debug, making them a suitable choice for smaller-scale problems (Sweetser & Wiles 2002). FSMs can be implemented as core modules of game objects without significant performance loss, making them a cost-effective solution in terms of technical resources. However, FSMs scale poorly as the complexity grows exponentially with the number of states and transitions, making them challenging to manage in larger-scale scenarios (Sweetser & Wiles 2002).

Behaviour Trees provide a comprehensive approach to AI functionality. The tree structure allows for logical decision-making by trying all combinations until a logical behaviour is identified or all options are exhausted. Behaviour Trees are more readable compared to complex FSMs as they organise specific behaviours at the top of the tree, progressively breaking them down into sub-trees of increasing specificity (Perez et al. 2011).

In summary, GOAP excels in handling contingencies and adaptability, FSMs are simple and suitable for smaller-scale problems, and Behaviour Trees offer completeness of functionality and readability. The choice of AI approach depends on the specific requirements of the game and the desired balance between complexity and simplicity.

4.2 Evaluation

When considering the suitability of the AI approaches for specific games, it becomes clear that each approach has its own strengths and weaknesses.

The most logical choice for utilising Goal-Oriented Action Planning (GOAP) is in the game *Hitman*. With its self-compensating architecture for decision making and event handling, GOAP can easily handle the complex behaviour decisions required for the game's world. In *Hitman*, numerous independent AIs need to make decisions based on dynamic factors like weapon possession, witnessing player actions, and being attacked. GOAP's ability to map game states to goals allows for streamlined AI behaviour, such as having the goal of killing the player while intelligently managing tasks like finding a weapon and

Electronic copy available at: https://ssrn.com/abstract=4513757

reacting differently to disguises. Additionally, GOAP ensures valid plans and maintains a self-maintained action base, making it the best fit for *Hitman*.

Finite-State Machines (FSMs) are most suitable for the game *Skyrim*. The game's open world and intricate web of non-playable character (NPC) interactions benefit from the simplicity and low technical cost of FSMs. NPCs in *Skyrim*, such as guards, shopkeepers, and enemies, can exhibit appropriate behaviours like patrolling, conversing, attracting customers, roaming, and fighting using FSMs. The small-scale complexity of FSMs brings these AIs to life without overburdening the hardware. However, FSMs do have limitations, as they cannot detect discrepancies between expected and actual game states.

For the game *DEFCON*, linking Behaviour Trees to the game mechanics is the most effective approach. Behaviour Trees excel in handling a vast range of game states and planning behaviours across multiple possible outcomes. In *DEFCON*, the enemy AI can logically plan behaviours based on factors like enemy fleet locations, the number of scanning units, and the DEFCON level. The Behaviour Trees traverse their logic circuits to ensure successful operations, and the exponential number of outcomes allows for seemingly infinite behaviours in the game world. This complexity can be managed without an overly large behavioural logic base. Hence, Behaviour Trees are the best fit for *DEFCON*.

In conclusion, considering the strengths and weaknesses of each AI approach, GOAP is well-suited for *Hitman*, FSMs are ideal for *Skyrim*, and Behaviour Trees are the most suitable choice for *DEFCON*.

5 Conclusion

In conclusion, each AI approach has its own set of strengths and weaknesses. GOAP offers the advantage of decoupling goal selection from execution, although it still relies on a simplified finite-state machine. Finite-state machines are relatively simple to program and can be applied to various scenarios, but their complexity grows exponentially as the number of states increases. Behaviour trees allow for tasks to be broken down into logical, atomic actions and queries, but their time complexity can increase as the tree traversal becomes more sparse.

Based on the evaluation of the AI approaches, it is recommended to implement GOAP in *Hitman*, finite-state machines in *Skyrim*, and behaviour trees in *DEFCON*. These games present specific challenges that can be effectively addressed by the respective AI methods, taking into account factors such as complexity, flexibility, and the nature of the game's mechanics.

By leveraging the strengths of each AI approach, developers can enhance the AI behaviour in these games, creating more immersive and engaging experiences for players.

Electronic copy available at: https://ssrn.com/abstract=4513757

References

Delay, C. (2006), *DEFCON*, Introversion Software.

Hart, P., Nilsson, N. & Raphael, B. (1968), 'A formal basis for the heuristic determination of minimum cost paths', *IEEE Transactions on Systems Science and Cybernetics* **4**(2), 100–107.
URL: *https://doi.org/10.1109/tssc.1968.300136*

Howard, T. (2011), *The Elder Scrolls V: Skyrim*, Bethesda Softworks.

Lim, C.-U., Baumgarten, R. & Colton, S. (2010), Evolving behaviour trees for the commercial game defcon, *in* 'European conference on the applications of evolutionary computation', Springer, pp. 100–110.

Nareyek, A. (2004), 'Ai in computer games', *Queue* **1**(10), 58.

Orkin, J. (2003), 'Applying goal-oriented action planning to games', *AI game programming wisdom* **2**, 217–228.

Perez, D., Nicolau, M., O'Neill, M. & Brabazon, A. (2011), Evolving behaviour trees for the mario ai competition using grammatical evolution, *in* 'European Conference on the Applications of Evolutionary Computation', Springer, pp. 123–132.

Sweetser, P. & Wiles, J. (2002), 'Current ai in games: A review', *Australian Journal of Intelligent Information Processing Systems* **8**(1), 24–42.

Vorsholt, J. (2016), *Hitman*, IO Interactive.

Weber, B., Mateas, M. & Jhala, A. (2010), 'Applying goal-driven autonomy to starcraft', *Expressive Intelligence Studio* .
URL: *http://alumni.soe.ucsc.edu/~bweber/pubs/gda_aiide2010.pdf*

Weik, M. H. (2001), *depth-first search*, Springer US, Boston, MA, pp. 387–387.
URL: *https://doi.org/10.1007/1-4020-0613-6_4747*

Electronic copy available at: https://ssrn.com/abstract=4513757